

GroupXX DESIGN CONCEPTS - LAB 11: COUPLING & COHESION ANALYSIS

ITSS Software Development - Capstone Project Design Review Use Cases Analyzed:
PayOrder (VietQR, PayPal, Order Integration, Product Management)

This document presents the system design analysis corresponding to each level of coupling and cohesion present in the AIMS/ITSS source code. Levels without concrete examples in the project have been omitted.

1. COUPLING

Coupling measures the degree of interdependence between modules/classes. In high-quality designs, the goal is to minimize coupling (Loose Coupling) so that changes in one module do not negatively impact others.

1.1. Control Coupling

Definition: Control Coupling occurs when one module passes parameters or tokens that explicitly direct the internal execution flow or decision-making branching inside another module.

Related Modules	Description	Improvement Direction
ProductService & ProductValidatorFactory	ProductService passes a string parameter mediaType to let ProductValidatorFactory decide which validator strategy to return.	This is an acceptable structure of the Factory Pattern to encapsulate the validator selection logic, keeping the service code clean and easy to extend.

1.2. Data Coupling

Definition: Data Coupling occurs when modules interact strictly through simple and independent data parameters (primitive values). This represents the ideal level of coupling.

Related Modules	Description	Improvement Direction
PaypalApiClient & PaypalService	Communicates by passing simple primitive parameters (orderId: number, usdAmount: string) to execute external API calls.	No changes required; this is the most ideal level of coupling in system design.
VietqrApiClient & VietqrPaymentService	Passes independent and simple primitive parameters (amount, content, orderId) to generate dynamic payment QR codes.	Maintain current function signatures with primitive data types.

PaymentRepository & PaypalRepository	The service layer only passes primitive parameters or minimalist transaction records to query the database.	Maintain the existing minimalist data boundaries.
---	---	---

2. COHESION

Cohesion measures the degree of focus and alignment of internal elements within a module or class. High cohesion (particularly Functional Cohesion) is the primary design goal.

2.1. Procedural Cohesion

Definition: Procedural Cohesion occurs when elements are grouped together because they execute in a specific, mandatory sequential order to achieve a business workflow.

Related Modules	Description	Improvement Direction
OrderService (placeOrder flow)	The placeOrder() method groups the steps of stock checking -> item reserving -> shipping fee calculating -> order creating -> invoice generating according to a mandatory business sequence.	Continue delegating the shipping fee calculation to ShippingCalculatorService to keep this class focused strictly on procedural workflows.
VietqrPaymentService (createPayment flow)	Executes the workflow as a sequence of steps: validating order status -> generating QR code -> recording the transaction record.	Delegate external API calls to the corresponding API Client to keep the service logic concise.

2.2. Communicational Cohesion

Definition: Communicational Cohesion occurs when all functions or methods within a module operate on, or produce, the exact same input or output data structures.

Related Modules	Description	Improvement Direction
VietqrRepository	All methods in the class operate on the same input/output data structure (the vietqr transactions data table).	Keep repository methods fully focused on their corresponding entity.
PaypalRepository	All queries and updates of the repository are directed toward operating on the paypal transactions entity.	Maintain the current design.

2.3. Sequential Cohesion

Definition: Sequential Cohesion occurs when the output of one step serves as the direct input to the next step in an execution sequence.

Related Modules	Description	Improvement Direction
VietqrPaymentService (Callback sync)	Webhook processing sequence: validating webhook data -> finding the transaction -> updating the payment status -> updating the order status (the output of the previous step serves as the input for the next).	Ensure comprehensive database transaction rollbacks are implemented if any step in the sequence encounters an error.

2.4. Functional Cohesion

Definition: Functional Cohesion occurs when all elements of a module or class cooperate to perform a single, well-defined task. This represents the highest and most desirable level of cohesion.

Related Modules	Description	Improvement Direction
PaypalApiClient	Performs a single, cohesive task of sending HTTP requests to communicate with the PayPal Sandbox system.	No changes required; the class achieves maximum functional cohesion.
ShippingCalculatorService	Focuses purely on performing one function: calculating shipping costs based on weight and province.	Keep the calculation logic independent of the system's larger entities.
ProductValidatorFactory	Only assumes the responsibility of instantiating and returning the appropriate validator strategy to check product data.	Maintain the single responsibility of this class.